Roteiro de fala: apresentação da Missão 3 (10 minutos)

SecureDocs / TechSecure · Criptografia simétrica · 5 alunos × 2 minutos (~250 palavras cada)

Aluno	Tempo	Slides	Tema	Passa a palavra com...
Aluno 1	0:00–2:00	1–4	Incidente, fundamentos, ECB × CBC	"...e o primeiro algoritmo que estudamos foi o DES."
Aluno 2	2:00–4:00	5–6	DES e ataque de força bruta	"...uma alternativa mais flexível é o RC5."
Aluno 3	4:00–6:00	7–8	RC5 e desempenho	"...mas o padrão mundial é o AES."
Aluno 4	6:00–8:00	9–11	AES, biblioteca, experimento	"...agora vamos comparar e decidir."
Aluno 5	8:00–10:00	12–14	Decisão, distribuição de chaves, conclusão	Abre para perguntas

Aluno 1 · Abertura, incidente e fundamentos

Slides 1 a 4 · 0:00–2:00

[Slides 1–2] "Boa tarde. Somos a equipe de segurança contratada pela TechSecure, e hoje apresentamos a Missão 3: milhares de documentos foram roubados. A apresentação foi dividida entre os cinco integrantes, como mostra o roteiro."

[Slide 3] "Um invasor copiou arquivos do servidor do SecureDocs. Eles estavam cifrados com cifras clássicas, como César e Vigenère. O problema é que essas cifras têm poucas chaves (a de César tem 25) e preservam a estatística da língua. Com análise de frequência, o invasor lê tudo. A empresa então perguntou: mesmo que roubem os arquivos, como impedir que leiam o conteúdo? A resposta é usar uma cifra simétrica moderna."

[Slide 4] "Na criptografia simétrica a mesma chave cifra e decifra. As cifras modernas trabalham em blocos e combinam confusão, com as S-boxes, e difusão, com as permutações, de modo que mudar um bit muda metade da saída. Mas só o algoritmo não basta: o modo de operação também importa. Esta imagem foi gerada pela nossa biblioteca. No modo ECB, o desenho do cadeado continua visível mesmo cifrado com AES. No modo CBC, vira ruído. Por isso usamos CBC com vetor de inicialização aleatório. E o primeiro algoritmo que estudamos foi o DES."

Aluno 2 · DES e o ataque de força bruta

Slides 5 e 6 · 2:00–4:00

[Slide 5] "O DES foi o padrão americano de 1977. Ele cifra blocos de 64 bits com uma chave de 64 bits, mas 8 são de paridade, então só 56 são efetivos. A estrutura é uma rede de Feistel com 16 rodadas: o bloco é dividido em duas metades, a metade direita passa pela função f e o resultado é combinado com a esquerda por XOR. Dentro da f há uma expansão de 32 para 48 bits, o XOR com a subchave, as 8 S-boxes, que são a única parte não linear, e uma permutação. A vantagem do Feistel é que decifrar é o mesmo circuito com as subchaves na ordem inversa. Implementamos tudo isso em Python, inclusive o 3DES."

[Slide 6] "Para provar que o DES é fraco, nós o atacamos. Como todo contrato começa com a palavra 'CONTRATO', o invasor tem um texto conhecido. Deixamos 14 bits da chave desconhecidos e nosso script encontrou a chave em 3 segundos, testando cerca de 3.800 chaves por segundo. Extrapolando para os 56 bits, uma máquina dedicada como a Deep Crack de 1998 leva 4,6 dias em média, e um cluster de GPUs atual, umas 10 horas. Em 1999 o DES foi quebrado em 22 horas. Já o AES, com a mesma velocidade, levaria 10^{57} anos. O DES está descartado, e o 3DES também, porque foi proibido pelo NIST desde 2024. Uma alternativa mais flexível é o RC5."

Aluno 3 · RC5 e desempenho

Slides 7 e 8 · 4:00–6:00

[Slide 7] "O RC5 foi criado por Ron Rivest em 1994. A grande característica dele é ser parametrizável: escolhemos o tamanho da palavra, o número de rodadas e o tamanho da chave. A versão de referência é a RC5-32/12/16, com bloco de 64 bits, 12 rodadas e chave de 128 bits. Ele usa só três operações: soma, XOR e rotação. O detalhe genial é que a rotação depende dos próprios dados. Isso dá a não linearidade sem precisar de S-boxes. A chave é expandida em uma tabela S, iniciada com constantes derivadas dos números e e φ. Nossa implementação aceita palavras de 16, 32 e 64 bits e passou nos vetores de teste originais do Rivest."

[Slide 8] "O RC5 é curto e rápido. Na nossa medição em Python ele foi o mais rápido, com 705 KB/s contra 316 do AES-128. Mas tem problemas sérios: com 12 rodadas ele é vulnerável a um ataque diferencial com 2^{44} textos escolhidos, o bloco de 64 bits tem o mesmo limite do DES, ele nunca foi padronizado pelo NIST e não tem aceleração em hardware. E em processadores reais o AES, com as instruções AES-NI, chega a vários gigabytes por segundo. O padrão mundial é o AES."

Aluno 4 · AES, biblioteca e experimento

Slides 9 a 11 · 6:00–8:00

[Slide 9] "O AES venceu um concurso público do NIST e é padrão desde 2001. O bloco é de 128 bits, organizado em uma matriz 4×4 de bytes, e a chave pode ter 128, 192 ou 256 bits, com 10, 12 ou 14 rodadas. Cada rodada tem quatro etapas: SubBytes, que dá a confusão; ShiftRows e MixColumns, que dão a difusão; e AddRoundKey, que mistura a chave. Na nossa implementação a S-box não foi copiada: ela é calculada no código com aritmética no corpo GF(2^8). Depois de 25 anos, o melhor ataque só reduz o AES-128 de 2^{128} para 2^{126}, sem efeito prático."

[Slide 10] "Tudo isso virou a biblioteca securedocs_crypto, em Python puro e sem dependências. Ela tem DES, 3DES, RC5, AES, os modos ECB, CBC e CTR, e cifragem de arquivos. Para provar que está correta, criamos 18 testes com os vetores oficiais do NIST e do Rivest, e todos passaram."

[Slide 11] "Este é o experimento pedido. Pegamos um contrato, ciframos com AES-256 e o arquivo virou bytes ilegíveis: o valor do contrato não aparece em lugar nenhum. Ao decifrar, o hash SHA-256 é idêntico ao original. Também incluímos um código de integridade HMAC: se o invasor alterar um único bit, o SecureDocs recusa o arquivo, o que resolve o caso do contrato adulterado. Agora vamos comparar e decidir."

Aluno 5 · Decisão, distribuição de chaves e conclusão

Slides 12 a 14 · 8:00–10:00

[Slide 12] "Colocando lado a lado: o DES tem chave de 56 bits e cai em horas. O RC5 tem ataque conhecido e não é padronizado. O AES tem bloco de 128 bits, chave de até 256 e nenhum ataque prático. Nossa escolha é o AES-256, porque ele protege também contra o futuro: um computador quântico, com o algoritmo de Grover, reduziria a segurança à metade, e o AES-256 ainda manteria 128 bits. Em produção, usaríamos o modo AES-256-GCM de uma biblioteca auditada."

[Slide 13] "Mas descobrimos um novo problema: como entregar a chave ao destinatário? Se ela for pela Internet, o invasor também a pega. E com 1.200 usuários seriam necessárias 720 mil chaves. Estudamos canais alternativos, servidores de chaves como o Kerberos e o Diffie-Hellman. Nossa resposta é o envelope digital: o documento é cifrado com uma chave AES aleatória, e essa chave é cifrada com a chave pública do destinatário. Só quem tem a chave privada abre. Certificados digitais evitam o ataque do homem do meio, e a troca híbrida X25519 com ML-KEM protege contra computadores quânticos."

[Slide 14] "Concluindo: as cifras clássicas e o DES não protegem mais, e o modo de operação importa tanto quanto o algoritmo. Entregamos uma biblioteca testada e a cifragem de arquivos com AES-256. Nas próximas missões, com criptografia assimétrica, hash e assinatura digital, vamos completar a arquitetura do SecureDocs. Obrigado! Ficamos à disposição para perguntas."

Perguntas prováveis e respostas curtas

Por que não usar o 3DES, se ele tem 112 bits? É 3× mais lento que o DES, usa bloco de 64 bits (vulnerável ao Sweet32 após ~32 GB) e o NIST proibiu seu uso para cifrar dados desde 2024.

Por que AES-256 e não AES-128? O AES-128 é seguro hoje, mas contratos podem precisar de sigilo por décadas. Contra o algoritmo de Grover (computação quântica), o AES-256 mantém 128 bits de segurança. O custo extra é só 4 rodadas (14 contra 10).

Qual a diferença entre CBC e GCM? O CBC só garante confidencialidade, por isso adicionamos o HMAC. O GCM é um modo AEAD: cifra e autentica ao mesmo tempo e é mais rápido. Em produção usaríamos o GCM.

Por que a biblioteca não deve ir para produção? Python puro é lento e não é protegido contra ataques de canal lateral (tempo de execução e cache). Ela serve para estudo; em produção usam-se bibliotecas auditadas como *cryptography* ou OpenSSL.

O que é o salt no PBKDF2? É um valor aleatório que entra na derivação da chave a partir da senha. Ele faz a mesma senha gerar chaves diferentes e impede tabelas pré-calculadas. As 200 mil iterações deixam lentos os ataques de dicionário.

O Diffie-Hellman sozinho resolve a distribuição de chaves? Não. Sem autenticação, um atacante no meio pode fazer uma troca com cada lado. Por isso ele é combinado com certificados e assinaturas digitais, como no TLS 1.3.

Dicas: ensaiem com cronômetro (2 minutos cada); não leiam os slides, usem este roteiro como guia; se der tempo, rodem ao vivo `python exemplos/demo_aes_arquivo.py` na pasta `biblioteca` durante o slide 11.